0.8 Java

0.8.1 Einleitung

- Wiederholung der Konzepte objektorientierter Programmierung
- diesmal am Beispiel Java
- Gegenüberstellung C++ – Java
- allgemeine Grundlagen zu Java

0.8.1.1 C++ vs. Java

In dieser Vorlesung werden die Konzepte prozeduraler und objektorientierter Programmierung anhand von *C++* und *Java* erläutert.

C++

- Entstanden aus C
- Compiler übersetzt in Maschinencode
- Ermöglicht hardwarenahe Programmierung
- Schnell

Java

- Neuentwicklung
- Compiler übersetzt in Bytecode, der in VM läuft
- Automatische Speicherverwaltung
- Verzicht auf Zeigertypen (dadurch weniger fehleranfällig bei Programmentwicklung)

© R.-R. Grigat 2003-2019

0.8.1.2 Wiederholung: „Hallo Welt“ in C++

C++

```
1  #include <iostream>
2  using namespace std;
3  int main()
4  {
5      cout << "Hello World!" << endl;
6      return 0;
7  }
```

C++

- `main`-Funktion außerhalb einer Klasse
- Name der Datei, die `main` enthält, prinzipiell egal
- Grundsätzlich **`int`** `main`, d. h. `main` liefert immer **`int`** zurück
- Es darf nur eine `main`-Funktion in einem Programm existieren

0.8.1.3 „Hallo Welt“ in Java

Java

```
1 //HelloWorldApp.java
2  public class HelloWorldApp
3  {
4      public static void main(String[] args)
5      {
6          System.out.println("Hello World!");
7      }
8  }
```

Java

- `main`-Funktion ist statische Funktion innerhalb einer Klasse
- Name der Datei, die `main` enthält, *muss* identisch sein zu Name der Klasse, die `main` enthält
- Grundsätzlich **`void`** `main`, d. h. `main` liefert *niemals* Rückgabewert
- Jede öffentliche Klasse kann eine `main`-Funktion enthalten

© R.-R. Grigat 2003-2019

0.8.2 Kontrollstrukturen

C++ und Java habe folgende Kontrollstrukturen gemeinsam:

- Sequentielle Komposition `{}`
- Alternativanweisung `if else`
  - Bedingte Anweisung `if`
  - Fallunterscheidung `switch`
- Schleife (Iteration)
  - `while`-Schleife
  - `do-while`-Schleife
  - `for`-Schleife

Die Syntax der Kontrollstrukturen ist in C++ und Java jeweils identisch.

0.8.3 Modularisierung

0.8.3.1 Vom Quelltext zum Programm

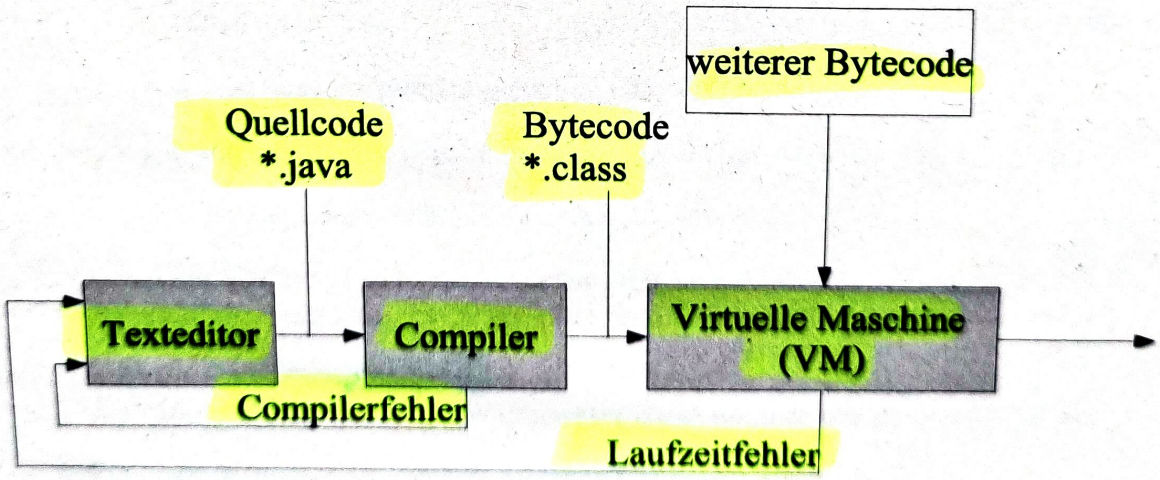


Abbildung 50: Java: Schritte vom Quelltext zum Programm

1. Kompilierung von `*.java`-Dateien zu `*.class`-Dateien

© R.-R. Grigat 2003-2019

2. Interpretierung der `*.class`-Dateien durch Virtual Machine

0.8.3.2 Kompilierung eines Javaprogramms

Der Compiler

- Jede Klasse aus einer `*.java`-Datei wird zu separater `*.class`-Datei kompiliert
- `*.class`-Dateien enthalten Bytecode, der durch Virtual Machine interpretiert wird
  - Falls `*.java`-Datei mehrere Klassendefinitionen enthält, werden auch mehrere `*.class`-Dateien erzeugt
  - Jede `*.java`-Datei darf aber nur eine öffentliche (**`public`**) Klasse enthalten
  - Bytecode kann nicht direkt von Betriebssystem ausgeführt werden

0.8.3.3 Interpretierung eines Javaprogramms

Die Virtual Machine

- Java-Programm besteht in Prinzip nur aus `*.class`-Dateien
- Mehrere `*.class`-Dateien können in einer `*.jar`-Datei (Java Archive) zusammengefasst (und komprimiert) werden
- `*.class` und `*.jar`-Dateien werden zur Laufzeit durch Virtual Machine interpretiert
- Suchpfade für diese Dateien werden über Systemvariable `CLASSPATH` gesetzt
- Aktuelles Verzeichnis „`.`“ muss explizit in `CLASSPATH` gesetzt werden

Start eines Javaprogramms

- Virtual Machine wird Name einer Klasse übergeben
- Aus dieser Klasse wird statische `main`-Methode ausgeführt

© R.-R. Grigat 2003-2019

0.8.3.4 Pakete in Java

Definition: Pakete (packages)

- `*.class`-Dateien werden in Paketen zusammengefasst
- Jede Klasse gehört zu genau einem Paket, Festlegung in erster Zeile der `*.java`-Datei mit Anweisung **`package`**
  - Beispiel: **`package`** `de.tuvision;` gibt an, dass alle Klassen der `*.java`-Datei Teil des Pakets `de.tuvision` sind
- Suchpfad schließt Paketnamen als Verzeichnisnamen ein
  - Klassen möglichst in gleichnamigem Verzeichnis ablegen
- Jede Klasse ist über qualifizierten Namen eindeutig ansprechbar
  - Beispiel: `java.lang.String` ist Klasse `String` aus dem Paket `java.lang`
- Konvention:
  - Namen von *Paketen* werden kleingeschrieben
  - Namen von *Klassen* beginnen mit Großbuchstaben

0.8.3.5 Importieren von Paketen

Problem:

- Klassen aus fremden Paketen müssen bekannt gemacht werden (falls nicht **`public`**)

Lösung

- **`import`** teilt Compiler die Namen einer oder mehrerer Klassen anderer Pakete mit
- *Jokerzeichen* `*` erlaubt
  - Beispiel: **`import`** `java.util.*`
  - macht alle Klassen aus dem Paket `java.util` bekannt
- Das Paket `java.lang` wird automatisch importiert (enthält `String`, `Math` usw.)

© R.-R. Grigat 2003-2019

0.8.3.6 Namensräume in Java

Problem

- Verschiedene Pakete könnten denselben Klassennamen verwenden

Übliche Lösung in Java:

- Domainnamen *in umgekehrter Reihenfolge* als voll qualifizierten Paketnamen verwenden

Beispiel

- Domain `tuvision.de`
- Name des Pakets *Beispiel*: `de.tuvision.beispiel`
- auf Rechner muss Verzeichnisstruktur `de\tuvision\beispiel` existieren
- z. B. `C:\javapakete\de\tuvision\beispiel`
- Pfad `C:\javapakete` der Systemvariablen `CLASSPATH` hinzufügen

0.8.4 Klassen in Java

- In Java keine Trennung von Klassendefinition und Methodendefinition
- Methoden werden immer innerhalb einer Klasse definiert
- Es gibt keine Headerdateien
- Klassendefinitionen können ihrerseits Klassendefinitionen enthalten (Nested Classes), diese können statisch oder nicht-statisch sein
  - Statische Nested Classes heißen *static nested* und haben *keinen* Zugriff auf Member der umschließenden Klasse
  - Nicht-statische Nested Classes heißen *innere Klassen* und haben Zugriff auf *alle* Member der umschließenden Klasse, private eingeschlossen
  - Innere Klassen können zusätzlich **`protected`** oder **`private`** sein
  - *Lokale innere Klassen* und *anonyme innere Klassen* sind Spezialfälle innerer Klassen mit Definition innerhalb einer Memberfunktion

© R.-R. Grigat 2003-2019

Sichtbarkeit	von Klassen(typ)	von Variablen (z. B. Objekten) und Methoden
kein Modifikator	im eigenen Paket(package private)	im eigenen Paket(package private)
public	öffentlich	öffentlich
private	nur inner class: sichtbar in äußerer Klasse	in eigener Klasse
protected	nur inner class: sichtbar wie protected Var.	in eigener Klasse, abgeleiteten Klassen, allen Memberfunktionen des eigenen Pakets
final	Vererbung verhindert	Konstanten, kein Überschreiben der Methode in Subklassen
abstract	abstrakte Klasse	abstrakte Methoden
static	nur static nested class	statische Variablen (Klassenvar.) statische Methoden (Klassenmeth.)

Beispiel: Klassendefinition

```
1  // Auto.java
2  package de.tuvision.autobeispiel;
3  public class Auto {
4    public Auto () {
5      farbe = "rot"; // Zuweisung
6    }
7    public Auto (String farbe) {
8      this.farbe = farbe; // Zuweisung
9                          // this: Referenz auf aktuelle Instanz,
10                         // noetig wegen Doppeldeutigkeit von farbe
11                         // (Parameter bzw. Instanzvariable)
12   }
13   public String farbe (){
14     return farbe;
15   }
16   public void lackiere (String farbe) {
17     this.farbe = farbe;
18   }
19   private String farbe; // Instanzvariable
20 }
```

© R.-R. Grigat 2003-2019

0.8.4.1 Der Konstruktor

Konstruktor

- wird bei Erzeugung eines Objektes aufgerufen
- erlaubt initiale Belegung von Variablen (Zuweisung)
  - Ohne explizite Initialisierung in der Deklaration (→ Definition) werden Membervariablen und statische Variablen *automatisch* initialisiert (0, \0, false, null). Deshalb handelt es sich bei der Belegung im Konstruktor um eine Zuweisung.
  - Dies gilt *nicht* für lokale Variablen
    - Lokale Variablen müssen vor der ersten Verwendung explizit initialisiert werden
    - ansonsten Fehlermeldung des Compilers
- Funktion ohne Rückgabewert
- Funktionsname identisch zu Klassenname
- Compiler stellt default-Konstruktor ohne Parameter zur Verfügung, falls kein eigener Konstruktor definiert ist
- Default-Werte als Argumente sind in Java nicht erlaubt

0.8.4.2 Der Garbage Collector

Garbage Collector

- zerstört alle Objekte automatisch
- gibt Speicherplatz automatisch frei
  - Gefahr von Speicherlecks reduziert
- wird von Java Virtual Machine gesteuert
- normalerweise kein Eingriff des Programmierers notwendig

Es entfällt die Notwendigkeit eines Destruktors.

- Es gibt *keinen* Destruktor in Java!

© R.-R. Grigat 2003-2019

0.8.5 Variablen und Objekte

0.8.5.1 Variablen

In Java wird unterschieden zwischen:

- *Grundtyp*-Variablen
  - z. B. **int**, **double**, **boolean**
  - in Variable wird Wert direkt gespeichert
- *Referenztyp*-Variablen (typsichere Zeiger)
  - alle Variablen, die nicht Grundtyp sind
    - Zeiger auf Klassen
    - Zeiger auf Felder (Arrays)
  - in Variable wird *Zeiger* auf die Daten gespeichert
  - Bezeichnung der Zeiger fälschlich als *Referenz*, in unserem Skript als Referenz(Java)

Kein Adressoperator oder Inhaltsoperator (ähnlich & bzw. * in C++)

Beispiel

```
1  //MyInt.java
2  package de.tuvision;
3  public class MyInt {
4    int x;
5    MyInt (int x) {
6      this.x = x;
7    }
8    int getValue (){
9      return x;
10   }
11   void setValue (int x){
12     this.x = x;
13   }
14   void verdoppeln (){
15     x = x * 2;
16   }
```

© R.-R. Grigat 2003-2019

```
18 public static void main (String[] args) {
19   int a = 3;
20   int b = a; // Grundtypen
21   a = a * 2; // b wird nicht verändert <===
22   MyInt c = new MyInt (3);
23   MyInt d = c;      // Referenz(Java)
24   c.verdoppeln (); // d wird verändert <===
25   System.out.print(" a=" + a);
26   System.out.print(" b=" + b);
27   System.out.print(" c=" + c.getValue());
28   System.out.print(" d=" + d.getValue());
29 }
30 } // Ende von MyInt
```

Ausgabe: a=6 b=3 c=6 d=6

0.8.5.2 Referenzen: C++ vs. Java

Achtung:

- Referenzen(C++) und Referenzen(Java) sind unterschiedliche Konzepte, Verwechslungsgefahr!
- Referenzen(Java) sind Zeiger
- Dereferenzierung: Punkt in Java entspricht -> in C++

Eigenschaft	Referenz(C++)	Referenz(Java)
Verweis enthält	Zielinhalt	Adresse
Veränderung der Zuordnung einer Referenz	nicht möglich	möglich
Referenz belegt Speicher	nein	ja
anschaulich:	„anderer Name“	„typsicherer Zeiger“

Tabelle 8: Referenz(C++) vs. Referenz(Java)

© R.-R. Grigat 2003-2019

0.8.5.3 Beispiel zu Referenzen in C++ und Java

Beispiel C++

```
//CMyInt.hpp
/* ... */
class CMyInt{
public:
    CMyInt(int xNew) {
      x = xNew;
    }
    int getValue() {
      return x;
    }
    void verdoppeln() {
      x = x * 2;
    }
private:
          int x;
};
```

```
int main(int argc, char *argv[])
{
  CMyInt a(1);
  CMyInt &b = a; //Referenz(C++)
        // b anderer Name für a
  CMyInt c(6);
  // a.x==1  b.x==1  c.x==6

  b = c;
  // entspricht:  a = c;
  // a.x==6  b.x==6  c.x==6

  c.verdoppeln();
  // a.x==6  b.x==6  c.x==12

  a.verdoppeln();
  // a.x==12  b.x==12  c.x==12
  return 0;
}
```

Beispiel Java

© R.-R. Grigat 2003-2019

```
1  //MyInt.java
2  package de.tuvision;
3  public class MyInt {
4
5    MyInt (int x) {
6      this.x = x;
7    }
8    int getValue (){
9      return x;
10   }
11   void verdoppeln (){
12     x = x * 2;
13   }
14   private int x;
15
16   // Fortsetzung s.u.
```

```
1  public static void main
2          (String[] args) {
3    MyInt a = new MyInt (1);
4    MyInt b = a;
5      // b ist Referenz auf a
6    MyInt c = new MyInt (6);
7    // a.x==1  b.x==1  c.x==6
8
9    b = c;
10   // b ist jetzt Referenz auf c
11   // a.x==1  b.x==6  c.x==6
12
13   c.verdoppeln ();
14   // a.x==1  b.x==12  c.x==12
15   a.verdoppeln ();
16
17  // a.x==2  b.x==12  c.x==12
18 } // main
19 } // MyInt
```

0.8.5.4 Vergleich von Variablen

Der Operator ==

- Bei *Grundtypen* wird der Wert der Variablen verglichen

© R.-R. Grigat 2003-2019

- Bei *Referenztypen* wird der Wert der *Referenz(Java)*, *nicht* der Wert der Membervariablen verglichen

Beispiel

```
1   int a = 3;                                  // Grundtyp
2   int b = a;                                  // Kopie von a
3   MyInt c = new MyInt (3);                    // Referenz(Java) bzw. Zeiger
4   MyInt d = c;                                // Referenz(Java) auf c kopieren
5   MyInt e = new MyInt (d.getValue ());// Referenz(Java) auf neues Objekt
6
7   System.out.println (a == b);//Ausgabe true, Vergleich der Werte
8   System.out.println (c == d);//Ausgabe true, Vergleich der Zeiger
9                               //c und d verweisen auf
10                              //denselben Speicherplatz
11  System.out.println (d == e);//Ausgabe false, Vergleich der Zeiger
12                              //e ist ein anderes Objekt als d
```

0.8.5.5 Vergleich von Referenztypen

Die Klasse `Object`

- In Java sind *alle Klassen* implizit von der Klasse `Object` abgeleitet
- In `Object` ist die Methode **`boolean`** `equals (Object o)` definiert
- `equals` vergleicht zunächst auch zwei Referenzen (also *nicht* den Inhalt)
- `equals` kann (und wird meistens) in abgeleiteten Klassen so überladen, dass der Inhalt verglichen wird

0.8.5.6 Beispiel zu `equals()`

Beispiel

```
1 //MyInt.java
2 package de.tuvision;
3 class MyInt {
4   int x;
5   MyInt (int x) {
```

© R.-R. Grigat 2003-2019

```
6       this.x = x;
7     }
8     int getValue (){
9       return this.x;
10    }
11    void setValue (int x){
12      this.x = x;
13      return;
14    }
15    public boolean equals (MyInt i){
16    // Methode equals() überladen,
17    // um Inhalte zu vergleichen
18      return (x == i.getValue ());
19    }  // Ende equals

22  public static void main(String[] args){
23    int a = 3;  // Grundtyp
24    int b = a;  // b ist Kopie von a
25
26    MyInt c = new MyInt(3);//Referenztyp
27    MyInt d = c;// d Kopie von Zeiger c
28    MyInt e = new MyInt(d.getValue ());
29                // e ist Zeiger
30                // auf neues Objekt -
31
32    System.out.println(a==b);
33      // true, Werte sind gleich
34    System.out.println(c==d);
35      // true, Zeiger sind gleich
36    System.out.println(c==e);
37      // false, Zeiger sind ungleich
38    System.out.println(c.equals (e));
39      // true, Inhalte sind gleich
40  } // Ende main
41  }   // Ende MyInt
```

0.8.5.7 Wrapper-Klassen für Grundtypen

Wrapper-Klassen

- Für jeden Grundtyp existiert Wrapper-*Klasse*

© R.-R. Grigat 2003-2019

- Beispiel: `Integer` für **`int`**, `Double` für **`double`** usw.
- Rechenoperationen mit Wrapper-Klassen *nicht* möglich

Anwendung von Wrapper-Klassen

- Konvertierung von Strings in Grundtyp-Werte
- Speicherung von Grundtypen in *Collections* (entspricht C++ Container)
  - Collections können nur Klassen speichern, keine Grundtypen

© R.-R. Grigat 2003-2019

Beispiel

```
1  LinkedList myIntList = new LinkedList (); // eine Collection...
2  myIntList.add (new Integer (0));   // ...kann nur Objekte enthalten
3                                     // add fügt Liste Element hinzu
4  Integer x = new Integer ("123");   // Initialisierung mit String
5  int a = x.intValue ();             // Konvertierung nach int
```

0.8.5.8 Erzeugen eines Objektes

Objekt

- Instanz einer Klasse
- muss mit **new** erzeugt werden
- wird immer in Referenztyp-Variablen gespeichert

Beispiel Java

```
1  Auto erstwagen = new Auto ();
2  Auto zweitwagen = new Auto ("gelb");
```

Achtung:

- Objekt ohne Parameter muss mit () erzeugt werden
- Unterschied zu C++

Beispiel C++

```
1  CAuto *auto1 = new CCabrio;
```

0.8.5.9 Felder (Arrays) in Java

Beispiel: Deklaration eines Arrays

```
1 Auto [] autos;
2 // Auto autos[];  ist aequivalent
```

© R.-R. Grigat 2003-2019

Beispiel: Initialisierung eines Arrays

```
1 Auto autos[];
2 autos = new Auto [5];  //Arrayelemente werden mit 0 initialisiert
3 //äquivalente Anweisung:
4 Auto autos[] = new Auto [5]
```

Ein Array wird mit **new** erzeugt.

Beispiel: Initalisierung eines Arrays mit Objekten

```
1 Auto mehrAutos[] = {new Auto("rot"), new Auto("gelb")};
2 //Elementanzahl wird aus Länge der Elementliste ermittelt
```

In Java findet Bereichsüberprüfung statt, Zugriff auf `autos[5]` führt zu Fehlermeldung `ArrayIndexOutOfBoundsException`

0.8.6 Methoden

Methode

- ersetzt nicht-objektorientierte Einheiten wie Unterprogramm, Prozeduren, Funktionen
- Methoden immer Teil einer Klasse
- Überladen von Methoden möglich
- Überschreiben von Methoden möglich
- Default-Argumente bei Methoden in Java nicht erlaubt

Parameterübergabe

- bei Grundtypen: *call by value*
- bei Referenztypen(Java): *call by value*
- *kein* call by reference in Java!

© R.-R. Grigat 2003-2019

0.8.6.1 Parameterübergabe: Grundtypen

Beispiel: Call by Value bei Grundtypen

```
1  //Verdoppler.java
2  package de.tuvision.beispiel;
3  public class Verdoppler {
4    public Verdoppler () {}
5    public int verdoppeln (int x) {
6      x = x * 2;
7      return x;
8    }
9    public static void main (String[] args) {
10     Verdoppler v = new Verdoppler ();
11     int a = 3;
12     int b = v.verdoppeln (a); // call by value
13     System.out.println (a + ", " + b);
14   }
15 }
```

- a und b sind Grundtypen
- Übergabe in Methoden als Kopie
- Ausgabe: 3, 6

0.8.6.2 Parameterübergabe: Referenztypen

Beispiel: Call by Value bei Referenztypen

```
1  //Verdoppler.java
2  package de.tuvision.beispiel;
3  class MyInt {
4    int x;
5    public MyInt(int x) {
6      this.x = x;
7    }
8    public int getValue(){
9      return this.x;
10   }
11   public void setValue(int x){
```

© R.-R. Grigat 2003-2019

```
12      this.x = x;
13    }
```

```
15 public class Verdoppler {
16   public MyInt verdoppeln (MyInt x) {
17      x.setValue(x.getValue()*2);
18      return x;
19    }
20    public static void main
21        (String[] args) {
22      Verdoppler v = new Verdoppler ();
23      MyInt c = new MyInt(3);
24      v.verdoppeln(c); //call by value
25      System.out.println (c.getValue());
26    }
27 }
```

- c ist Referenztypvariable, Referenz(Java), Zeiger
- Übergabe in Methoden als flache Kopie eines Zeigers
- Ausgabe: 6

0.8.6.3 Statische Methoden in Java

Problem:

- Methode `verdoppeln` greift auf keine Variablen der Klasse `Verdoppler` zu
- Erzeugen eines Objekts `v` vom Typ `Verdoppler` erscheint umständlich und wenig sinnvoll

Lösung:

- Statische Methode verwenden
- Achtung: Statische Methoden können nicht auf nicht-statische Membervariablen zugreifen (ebenso wie in C++)

© R.-R. Grigat 2003-2019

0.8.6.4 Statische Variablen und Methoden: Beispiel Java

Beispiel

```
1  //Auto.java
2  package de.tuvision.autobeispiel;
3  public class Auto {
4    public Auto () {
5      this.farbe = "rot"; // Initialisierung
6    }
7
8    public static void setTacho_kmph(boolean x){
9      tacho_kmph = x;
10   }
11   private static boolean tacho_kmph = true;
12       // Initialisierung direkt in Klassendefinition,
13       // in C++: Initialisierung immer außerhalb Klassendef.
14   private String farbe;
15
16   public static void main (String[] args) {
17     Auto.setTacho_kmph(false);  // noch kein Objekt erzeugt
18                                 //in C++: Auto::setTacho_kmph(false);
19     Auto erstwagen = new Auto ();
20   }
21 }
```

0.8.7 Komposition

Beispiel: Auto „hat eine“ Hupe

```
1  //Hupe.java
2  package de.tuvision.autobeispiel;
3  public class Hupe {
4    public Hupe
5        (String ton, int staerke) {
6      this.ton = ton;
7      this.staerke = staerke;
8    }
9    public void hupen (int laenge) {
10     /* ... */
11   }
12   private String ton;
```

© R.-R. Grigat 2003-2019

```
13   private int staerke;
14 }
```

```
1  //Auto.java
2  package de.tuvision.autobeispiel;
3  public class Auto {
4
5    public Auto () {
6      this.farbe = "rot";
7    }
8    public Hupe hupe
9        = new Hupe("hohes C", 128);
10   private String farbe;
11 }
```

0.8.8 Vererbung

Beispiel

```
1  //Auto.java
2  package de.tuvision.autobeispiel;
3
4  public class Auto {
5
6    public Auto () {
7      this.farbe = "rot";
8    }
9    Hupe hupe
10       = new Hupe("hohes C", 128);
11
12   private String farbe;
13 }
```

```
1  //Cabrio.java
2  package de.tuvision.autobeispiel;
3
4  public class Cabrio extends Auto {
5    public Cabrio () {
6      /* ... */
7    }
8  }
```

© R.-R. Grigat 2003-2019

```
1 //AutoApp.java
2 Cabrio sportwagen = new Cabrio();
3 sportwagen.hupe.hupen (7);
```

- Vererbung mit Schlüsselwort **`extends`**
- Cabrio *„ist ein“* Auto
- Hupe wird vererbt auf Cabrio

0.8.8.1 Konstruktoren und Vererbung

- Im Konstruktor einer *abgeleiteten* Klasse wird stets als erstes ein Konstruktor der *Basisklasse* aufgerufen
  - entweder explizit Konstruktor mit Signatur nach Wahl,
  - ansonsten *implizit* der parameterlose Konstruktor (der dazu existieren muss, ansonsten Fehlermeldung des Compilers!)
- Dies wird rekursiv fortgesetzt bis zum Aufruf des Konstruktors `Object()`. Konstruktoren der Hierarchie werden also Top→Down abgearbeitet, `Object()` zu Beginn.
- Konstruktoren der Basisklassen brauchen in Java also nicht vererbt werden (anders als alle anderen Methoden)
- Expliziter Aufruf eines Konstruktors der Basisklasse mit Parametern a, b, c erfolgt mittels **`super()`**, also z. B. **`super(a, b, c)`**
- Wird (nur) parameterbehafteter Konstruktor in Basisklasse definiert, so existiert dort kein Default-Konstruktor mehr

0.8.8.2 Konstruktoren und Vererbung (2)

Beispiel

© R.-R. Grigat 2003-2019

```java
// Auto.java
package de.tuvision.autobeispiel;
public class Auto {
 public Auto () {
    this.farbe = "rot";
  }
  public Auto (String farbe) {
    this.farbe = farbe;
  }
private String farbe;
}
```

```java
// Cabrio.java
package de.tuvision.autobeispiel;
public class Cabrio extends Auto {
  public Cabrio () {}
      //impliziter Aufruf von Auto()
  public Cabrio (String farbe) {
    super (farbe);
      //expliziter Aufruf
      //des Konstruktors Auto(farbe)
  }
}
```

0.8.8.3 Zugriffsrechte

Modifikatoren für *Klassen*

- **public**: Klasse ist überall sichtbar
- ohne Modifikator (package private): Klasse ist nur innerhalb des eigenen Pakets sichtbar
- Es gibt *keine* **private** oder **protected** top-level Klassen
- In *inneren Klassen* sind **private** oder **protected** zulässig

Modifikatoren für *Variablen und Methoden*

- **public**: Variablen und Methoden in allen Klassen sichtbar

© R.-R. Grigat 2003-2019

- **private**: Variablen und Methoden nur innerhalb (aller Objekte) der eigenen Klasse sichtbar
- **protected**: Variablen und Methoden in eigener Klasse, allen ihren Subklassen, und in allen Methoden des eigenen Pakets sichtbar
- ohne Modifikator (package private): Variablen und Methoden in allen Klassen des eigenen Pakets sichtbar

Beispiel

```
1  //Hupe.java
2  package de.tuvision.autobeispiel;
3  class Hupe {          // nicht public
4        //nur innerhalb Paket sichtbar
5    Hupe (String ton, int staerke) {
6      this.ton = ton;
7      this.staerke = staerke;
8    }
9    void hupen (int laenge) {
10   /* ... */
11   }
12   private String ton;
13         //nur in Hupe sichbar
14   private int staerke;
15 }
```

```
1  //Auto.java
2  package de.tuvision.autobeispiel;
3  public class Auto {
4                //überall sichtbar
5     public Auto () {
6     this.farbe = "rot";
7    }
8    public void hupen(int laenge){
9      this.hupe.hupen (laenge);
10   }
11   private Hupe hupe
12       = new Hupe("hohes C", 128);
13   private String farbe;
14 }
```

© R.-R. Grigat 2003-2019

- Klasse Hupe nur innerhalb des Pakets `de.tuvision.autobeispiel` sichtbar
- vgl. **friend** in C++: explizit Zugriff für Klassen erlauben

0.8.8.4 Überschreiben von Methoden

Definition: Überschreiben von Methoden

- In abgeleiteter Klasse existiert Methodendefinition mit gleichem Namen und gleicher Signatur wie in Basisklasse
- Methode aus Basisklasse kann von außen über die abgeleitete Klasse nicht aufgerufen werden (sie ist überschrieben)
- in überschreibender Methode kann Methode der Basisklasse über **super** aufgerufen werden, z. B. `super.lackiere("rot")`
- **super** ist Referenz auf aktuelle Instanz, und zwar auf Teilelement der unmittelbar übergeordneten Basisklasse
- Hintereinanderausführung (z. B. `super.super.lackiere()`) bei mehrfacher Vererbung ist *nicht* möglich

Beispiel

```
1  //Auto.java
2  package de.tuvision.autobeispiel;
3
4  public class Auto {
5
6    public Auto () {
7      this.farbe = "rot";
8    }
9    public void lackiere (String farbe)
10   {
11     this.farbe = farbe;
12   }
13   private String farbe;
14 }
```

© R.-R. Grigat 2003-2019

```java
//Cabrio.java
package de.tuvision.autobeispiel;

public class Cabrio extends Auto {

  public Cabrio () {}

  public void lackiere (String farbe)
  {
    System.out.println
        ("Dach mit Folie schützen");
    super.lackiere (farbe);
        // ruft Auto.lackiere(farbe)
        // der aktuellen Instanz auf
  }
}
```

0.8.8.5 final

Das Schlüsselwort `final`

- bei Klassen: weitere Vererbung wird verhindert
- bei Methoden: Überschreiben wird verhindert
  - bei Java sind alle Methoden standardmäßig virtuell
  - zur Erinnerung: in C++ macht erst das Schlüsselwort **`virtual`** eine Methode virtuell
  - **`final`** verhindert das Überschreiben in Java
- bei Variablen: Konstante
  - Veränderung des Wertes wird verhindert
  - Bei konstanter Referenz(Java) ist nur dieser Zeiger konstant
  - Referenzierter Inhalt weiter veränderbar (außer referenziertes Objekt verhindert dies, wie z. B. Klasse String)

Beispiel

© R.-R. Grigat 2003-2019

```java
// Auto.java
package de.tuvision.autobeispiel;
public class Auto{
 public Auto () {  }

  public final void hupen (int laenge) throws HupeException{
        //Ueberschreiben von void hupen (int) verhindern
      this.hupe.hupen (laenge);
  }

private Hupe hupe = new Hupe ("hohes C", 128);
private final String farbe = "rot";
   // farbe ist jetzt konstant
}
```

0.8.8.6 Abstrakte Klasse

Definition:

- Definiert Schnittstelle (Abschnitt 0.3.1)
- Instanzen *nur* für hiervon abgeleitete Klassen erlaubt
- In Java Realisierung mit Schlüsselwort **abstract**
- *Kann* abstrakte Methoden (d. h. Methoden ohne Implementierung) enthalten
- Kann auch konkrete Implementierung einzelner Methoden oder Variablen erhalten

Beispiel Java

```java
// Auto.java
package de.tuvision.autobeispiel;
public abstract class Auto {  // Abstrakte Klasse
  public void lackiere (String farbe) {
    this.farbe = farbe;
  }
  public abstract void waschen(); //Abstrakte Methode
  private String farbe;
}
```

© R.-R. Grigat 2003-2019

0.8.8.7 Interfaces (Schnittstellen)

Definition `interface`:

- Zulässig sind nur
  - abstrakte Methoden (d.h. Methoden *ohne* Implementierung)
  - Konstanten
- alle Methoden innerhalb eines Interfaces sind bereits *implizit* **`public`** und **`abstract`**
  - *keine* konkreten Implementierungen von Methoden erlaubt
- alle Variablen innerhalb eines Interfaces sind bereits *implizit* **`static`**, **`final`** (konstant) und **`public`**

Beispiel

```
1 public interface Auto {
2   void lackiere (String farbe);
3 }
```

0.8.8.8 Interface implementieren

Beispiel

```
1 public class Cabrio implements Auto {
2   public Cabrio () {
3     this.farbe="rot";
4   }
5   public void lackiere (String farbe) {
6     this.farbe = farbe;
7   }
8   private String farbe;
9 }
```

konkrete Klasse

- muss alle abstrakten Methoden überschreiben
- kann mehrere Interfaces implementieren
- kann gleichzeitig von (einer) Klasse erben

© R.-R. Grigat 2003-2019

Beispiel zu mehreren Interfaces und Vererbung

```
public class Cabrio extends Fahrzeug implements Auto, Spielzeug {/* ... */}
```

0.8.9 Polymorphie

Beispiel

```
// Auto.java
package de.tuvision.autobeispiel;

public interface Auto {
  void waschen ();
}
```

```
//Bus.java
package de.tuvision.autobeispiel;

public class Bus implements Auto {
   public Bus () {}

   public void waschen () {
    System.out.println("wasche Bus");
   }
}
// Polymorphie auch bei Vererbung
// von Basisklasse Auto möglich,
// z.B. Bus extends Auto
```

```
// Cabrio.java
package de.tuvision.autobeispiel;

public class Cabrio implements Auto {
   public Cabrio () {}

   public void waschen () {
    System.out.println("wasche Cabrio");
   }
}
```

```
// AutoApp.java
import de.tuvision.autobeispiel.*;
```

© R.-R. Grigat 2003-2019

```
3 /* ... */
4     Auto auto1 = new Cabrio();
5     Auto auto2 = new Bus();
6     auto1.waschen();
7     auto2.waschen();
```

0.8.10 Ausnahmen (Exceptions)

0.8.10.1 Werfen einer Ausnahme

Werfen einer Ausnahme in Java

- Werfen einer Ausnahme erfolgt mit Anweisung **throw**
- Wie alle Objekte muss auch Ausnahme mit **new** erzeugt werden
- Alle Ausnahmen werden von Klasse `Exception` abgeleitet

Beispiel

```
1 if(t == null)
2   throw new NullPointerException();
```

0.8.10.2 Fangen einer Ausnahme

Fangen einer Ausnahme in Java

- Fangen einer Ausnahme erfolgt mit Hilfe der Java-Anweisungen **try**`{Anweisungsblock}` und **catch**
- Zusätzlich existiert in Java optionaler **finally**-Block
- **finally** wird ausgeführt,
  - wenn alle Anweisungen im **try**-Block erfolgreich ausgeführt wurden, oder
  - wenn der entsprechende **catch**-Block wieder verlassen wird (auch durch z. B. erneutes **throw**)

© R.-R. Grigat 2003-2019

0.8.10.3 Beispiel zu try-catch-finally

Beispiel

```
1  void hupen (int laenge)  {
2    try{
3      if (laenge > 5) throw new Exception ("zu lang ");
4        /* ... */
5    }
6    catch (Exception e){
7      System.err.println("Fehler in Hupe: " + e.getMessage());
8    }
9    finally{  // wird in jedem Fall ausgeführt
10     System.out.println("Hupen beendet");
11   }
12 }
```

Exception

- Basisklasse für alle Ausnahmen
- Methode `getMessage()` liefert String „zu lang“, der dem Konstruktor übergeben wurde

0.8.10.4 Benutzung verschiedener Ausnahmetypen

Werfen unterschiedlicher Ausnahmetypen

- Auch in Java sind mehrere **`catch`**-Blöcke erlaubt
- Es wird *nur der erste* **`catch`**-Block ausgeführt, dessen Parametertyp mit der geworfenen Ausnahme übereinstimmt
- Alle Ausnahmen sind von der Klasse `Exception` abgeleitet und werden von **`catch`** `(Exception e)` gefangen
- Alle kritischen Laufzeitfehler (die nicht sicher gefangen werden können) sind von der Klasse `Error` abgeleitet und sollten nicht gefangen werden
- Klasse `Exception` und Klasse `Error` sind beide von Basisklasse `Throwable` abgeleitet

© R.-R. Grigat 2003-2019

0.8.10.5 Definition eigener Fehlerklassen

Eigene Fehlerklassen

- müssen von der Basisklasse `Exception` abgeleitet werden
- mit **`super`** kann der Konstruktor von `Exception` explizit aufgerufen und Fehlermeldung übergeben werden
- über Methode `getMessage()` (definiert in `Exception`) kann diese Fehlermeldung dann ausgewertet werden

Beispiel

```
1 // HupeException.java
2 package de.tuvision.autobeispiel;
3 public class HupeException extends Exception{
4   public HupeException () {}
5   public HupeException (String meldung){
6     super (meldung); // expliziter Aufruf
7     }                // des Konstruktors von Exception
8 }
```

0.8.10.6 Weiterreichen von Ausnahmen

- Ausnahmen können von innerhalb einer Methode nach außen weitergereicht werden
- Mit **`throws`** werden in der Methodendeklaration die Ausnahmeklassen angegeben, die innerhalb der Methode geworfen und nach außen weitergereicht werden können
- Aufruf einer Methode, in deren Methodendeklaration mit **`throws`** Ausnahmeklassen benannt sind, *muss* entweder
  - in **`try`**-Block stehen (d.h. Weiterreichen terminiert hier), oder
  - ihrerseits innerhalb einer äußeren Methode stehen, in deren Methodendeklaration mit **`throws`** dieselben Ausnahmeklassen benannt sind (rekursive Fortsetzung des Weiterreichens der Ausnahmen)

© R.-R. Grigat 2003-2019

0.8.10.7 Beispiel: try-Block in aufrufender Ebene und throws - throw

Beispiel

```
class Hupe {
  void hupen (int laenge) throws HupeException {
      if (laenge > 5) throw new HupeException ("zu lang ");
      /* ... */     // nicht verwechseln: throws und throw
  }
}
```

```
class Auto {
  public void hupenAuto (int laenge) {
   try{   // Fall 1: throws in hupen, try in Stufe darüber
      this.hupe.hupen (laenge);    // this. kann auch weggelassen werden
   }
   catch (HupeException e){
     System.err.println ("Hupenfehler: " + e.getMessage ());
   }
  }
  private Hupe hupe = new Hupe ("hohes C", 128);
}
```

0.8.10.8 Beispiel: throws in aufrufender Ebene und throws - throw

Alternative zu Beispiel

```
class Hupe {
  void hupen (int laenge) throws HupeException {   //aufgerufene Methode
      if (laenge > 5) throw new HupeException ("zu lang ");
      /* ... */
  }
}
```

```
class Auto {  // Fall 2: throws in hupen und auch in hupenAuto darüber
  public void hupenAuto (int laenge) throws HupeException{
                  // aufrufende Methode
                  // Compilerfehler, falls throws HupeException fehlt
                  // try-Block weiter außerhalb erforderlich
    this.hupe.hupen (laenge); // Aufruf der Methode hupen,
                              // darin wird evtl. HupeException geworfen
  }
  private Hupe hupe = new Hupe ("hohes C", 128);
}
```

© R.-R. Grigat 2003-2019

0.8.11 Objekte kopieren

Die Klasse `Object`

- Basisklasse für alle Klassen in Java
- enthält u. a. folgende Methode:
  `protected` `Object clone()`: Kopie eines Objekts erzeugen
- Methode kann in eigener Klassen überschrieben werden

0.8.11.1 Überschreiben der Methode `clone()`

`clone()`

- liefert Objekt vom Typ `Object` zurück
- ist in `Object` als **`protected`** deklariert, sollte also **`public`** überschrieben werden (Rückgabetyp bleibt `Object` !)
- Damit `clone()` erlaubt ist, muss die Klasse das Interface `Cloneable` implementieren
  - Das Interface ist leer, es enthält keine Methoden-Deklarationen oder Konstanten
  - Das Interface teilt dem Compiler mit, dass Instanzen der Klasse kopiert werden können
  - Exception `CloneNotSupportedException` wird geworfen, falls das Interface `Cloneable` nicht implementiert ist
- Aufruf von **`super`**`.clone()` reserviert Speicher für Kopie

Beispiel

```
1 public class Auto implements Cloneable{  // zusätzlich wäre
2 /* ... */                                // "extends Basisklasse" möglich
3   public Object clone (){  // überschreiben, Rückgabetyp muss Object bleiben
4     Object o = null;
5     try {
6       o = super.clone ();  // Basisklasse ist Object, nicht das Interface
7       // hier können weitere Anweisungen für Kopiervorgang stehen
8     }
```

© R.-R. Grigat 2003-2019

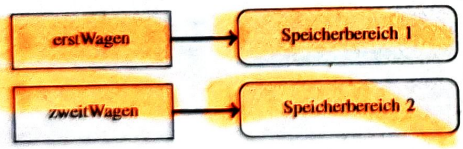
```
9     catch (CloneNotSupportedException e){
10      System.err.println ("clone not supported");  }
11    return o; } }  // end clone, end Auto
```

```
1   Auto erstwagen = new Auto ();
2   Auto zweitwagen = (Auto) erstwagen.clone (); //clone liefert Typ Object
```

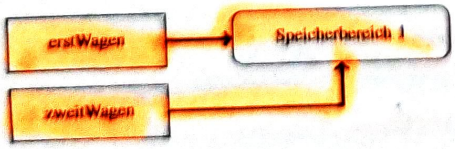
- Kopie von `erstwagen` erstellt
- Rückgabetyp `Object` muss in `Auto` gecastet werden
- Ein Interface (hier Cloneable) gilt nicht als Basisklasse
- Interfaceklassen sind nicht von Klasse Object abgeleitet!

Beispiel

```
1 // C++ - Impliziter Aufruf des
2 // Kopierkonstruktors
3 Auto erstWagen;
4 Auto zweitWagen = erstWagen;
```

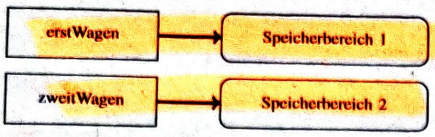



```
1 // Java - Zuweisung einer Referenz.
2
3 Auto erstWagen = new Auto();
4 Auto zweitWagen = erstWagen;
```




```
1 // Java clone - Reservierung neuen Speichers.
2 Auto erstWagen = new Auto();
3 // clone gibt ein Objekt vom Typ Object zurück und muss gecastet werden.
4 Auto zweitWagen = (Auto)erstWagen.clone();
```

© R.-R. Grigat 2003-2019



Hinweis zur erstellten Kopie:

- Sowohl in C++ (Kopierkonstruktor), als auch in Java (clone) werden nur 'flache' Kopien erzeugt.
- D.h. es werden nur Adressen, die auf Speicherbereiche zeigen kopiert und nicht der Speicherbereich selbst. Dies betrifft
  - in C++ Zeiger und
  - in Java Referenzen(Java), also ebenfalls Zeiger, als Membervariable.

0.8.12 Generische Programmierung in Java

Einleitung

- Seit JDK 1.5 generische Programmierung in Java möglich
- mit Hilfe von *Generics* (nicht verwechseln mit C++ Templates)
- An dieser Stelle nur kurze Einführung
- Ein gutes Tutorial: Gilad Bracha, Generics in the Java Programming Language (2004), `java.sun.com/j2se/1.5/pdf/generics-tutorial.pdf`
- Java Virtual Machine 1.5 oder größer wird zur Ausführung von Programmen mit Generics benötigt

0.8.12.1 Beispiel: Verkette Liste von Autos

Beispiel ohne Generics

```
1 List autoListe = new LinkedList(); // LinkedList abgeleitet von List
2 autoListe.add (new Auto());
3 Auto erstwagen = (Auto) autoListe.iterator().next(); // Cast zwingend
```

- Alle Collections (entsprechen Container in C++) speichern stets Typ `Object`
- Explizite Typumwandlung von `Object` nach `Auto` notwendig

© R.-R. Grigat 2003-2019

- Methode `iterator()` erzeugt `Iterator`-Objekt

Beispiel mit Generics

```
1  List<Auto> autoListe = new LinkedList<Auto>();
2  autoListe.add (new Auto ());
3  Auto x = autoListe.iterator().next();                    // Kein Cast nötig
```

- `List<Auto>`: Interface in Java, „Liste vom Typ Auto“
- Explizite Typumwandlung *nicht* notwendig

0.8.12.2 Definition von Generics

Beispiel: Liste

```
1 public interface List<E> {
2   void add (E x);
3   Iterator<E> iterator ();
4 }
```

```
1 public interface Iterator<E> {
2   E next ();
3   boolean hasNext ();
4 }
```

- `List<Auto>` erzeugt *keinen* zusätzlichen Programmcode, in dem <E> durch <Auto> ersetzt wird
- Jede Definition einer Generics-Klasse existiert genau einmal
  - Unterschied zu C++ Templates
- Konvention: Für Collections wird oft <E> als Typparameter verwendet (steht für Element)

© R.-R. Grigat 2003-2019

0.8.12.3 Generics und abgeleitete Klassen

Beispiel: Sind folgende Codezeilen erlaubt?

```
1  List<Auto> autoListe = new LinkedList<Auto>();
2  List<Object> objectListe = autoListe; // Fehler!
```

- Zeile 1: erlaubt, `List<E>` ist Basisklasse zu `LinkedList<E>`
- Zeile 2: *nicht* erlaubt, obwohl `Object` Basisklasse zu `Auto`, denn `List<Object>` ist keine Basisklasse zu `List<Auto>`

Angenommen, Zeile 2 wäre erlaubt:

```
3  objectListe.add(new Object());
```

- dürfte nun jede Instanz von `Object` oder einer abgeleiteten Klasse von `Object` einer `autoListe` hinzufügen
- erlaubt wären aber nur `Auto` und abgeleitete Klassen von `Auto`
- Compiler gibt Fehlermeldung in Zeile 2

0.8.12.4 Unbekannte Typen

Problem:
Wie schreibt man Methoden für beliebige Typen?

Beispiel: so nicht

```
1  void printListe (List<Object> liste) {
2   Iterator i = liste.iterator ();
3   for (int k = 0; k < liste.size (); k++) {
4     System.out.println (i.next ());
5   }
6  }
```

- `List<Object>` ist keine Basisklasse von z. B. `List<Auto>`

© R.-R. Grigat 2003-2019

0.8.12.5 Wildcards

Lösung
Java bietet *Wildcards*

Beispiel mit Wildcards

```
1  void printListe (List<?> liste) {
2    Iterator i = liste.iterator ();
3    for (int k = 0; k < liste.size (); k++) {
4      System.out.println (i.next ());
5    }
6  }
```

- `List<?>`: Liste von unbekannten Datentypen

0.8.12.6 Beispiel: Beliebige Collections anzeigen

Beispiel: allgemeine Ausgabemethode für Collections

```
1  void printCollection (Collection<?> c) {
2    Iterator i = c.iterator ();
3    for (int k = 0; k < c.size (); k++) {
4      System.out.println (i.next ());
5    }
6  }
```

- Methode kann von allen Klassen, die von `Collection` abgeleitet sind, aufgerufen werden
- Collections können beliebigen Referenztyp als Parameter enthalten (keine Grundtypen).
- Mehrfachimplementierung weder für verschiedene Typen von Collections noch für verschiedene Datentypen notwendig

© R.-R. Grigat 2003-2019

0.8.12.7 Einschub: Erweiterte for-Schleife seit Java 1.5

Beispiel mit erweiterter for-Schleife

```
1  void printCollection (Collection<?> c) {
2    for(Object e : c){
3      System.out.println(e);
4    }
5  }
```

- `for(Object e : c)` „Für alle e aus c“
- Notwendigkeit einer Zählvariable entfällt

0.8.12.8 Wildcards und Typsicherheit

Wildcards

- stehen für „unbekannter Typ“
- alle Klassen sind von `Object` abgeleitet
- Es ist erlaubt, Elemente aus `Collection<?>` in Variablen vom Typ `Object` abzulegen

Beispiel: Folgendes ist nicht erlaubt

```
1 Collection<?> c = new LinkedList<Auto>();  // <?> wird <Auto>
2 c.add( new Object() );  // Compilerfehler
```

- Es würde versucht, ein `Object` in einer Liste vom Typ `Auto` zu speichern
- `Auto` ist aber keine Basisklasse von `Object`

Fazit: Wildcards sind *typsicher*.

© R.-R. Grigat 2003-2019

0.8.12.9 Generics und Klassenhierarchien

Problem:
Wie schreibt man Methode für Basisklasse und davon abgeleitete Klassen?

Beispiel

```
1  public void alleWaschen (List<Auto> autos) {
2    for (Auto a: autos) {
3      a.waschen();
4    }
5  }
```

- Methode `alleWaschen` kann nur für `List<Auto>` aufgerufen werden
- `alleWaschen(List<Cabrio>);` ist nicht erlaubt
- `List<Auto>` ist keine Basisklasse zu `List<Cabrio>`

0.8.12.10 Beschränkte (bounded) Wildcards

Lösung:
Java bietet *beschränkte (bounded) Wildcards*

Beispiel mit beschränkten Wildcards

```
1  public void alleWaschen (List<? extends Auto> autos) {
2    for (Auto a: autos) {
3      a.waschen ();
4    }
5  }
```

- `List<? extends Auto>` „unbekannter Typ, der von `Auto` (direkt) abgeleitet ist“
- `Auto` ist sozusagen die obere Schranke von „unbekannter Typ“
- `alleWaschen(List<Cabrio>);` ist jetzt erlaubt
- `alleWaschen(List<Auto>);` ist auch erlaubt

© R.-R. Grigat 2003-2019

0.8.12.11 Einschränkung bei der Benutzung von beschränkten Wildcards

Beispiel: Folgendes ist nicht erlaubt

```
1  public void addCabrios (List<? extends Auto> autos) {
2    autos.add(new Cabrio());
3  }
```

- `List<? extends Auto>`: Liste mit Elementen vom Typ `Auto` oder beliebiger Unterklasse
- könnte also auch `Liste<Bus>` sein
- `Cabrio` kann aber nicht in einer `List<Bus>` gespeichert werden, da `Bus` keine Basisklasse zu `Cabrio` ist
- Hier muss `(List<Cabrio> autos)` als Parametertyp angegeben werden

0.8.12.12 Generische Methoden

Problem:
Objekte sollen von Array in Liste kopiert werden

Beispiel: so nicht

```
1  static void vonArrayNachList (Object [] array, List<?> liste) {
2    for (Object o : array) {
3      liste.add (o);
4  } }
```

- Elemente vom Typ `Object` können nicht in `List<?>` (Liste mit unbekanntem Typ) gespeichert werden

Lösung: ***Generische Methoden***

Beispiel

```
1  static <T> void vonArrayNachList (T [] array, List<T> liste) {
2    for (T o : array) {
3      liste.add (o);
4  } }
```

© R.-R. Grigat 2003-2019

0.8.12.13 Hinweise generische Methoden

Hinweis zu **static** in diesem Kontext:

- Generische Methoden müssen nicht **static** sein.
- vonArrayNachList ist hier **static** da die Methode eine Funktion erfüllt, die nicht an eine konkrete Instanz eines Objekts gebunden ist.
- Erinnerung: In Java können statische Methoden nur auf statische Member einer Klasse zugreifen.

0.8.12.14 Typparameter und generische Methoden

Generische Methoden

- Typparameter wird implizit aus Übergabetyp ermittelt
- Typparameter wird *nicht* explizit angegeben
- Compilerfehler, falls Typ nicht aufgelöst werden kann

Beispiel

```
1  static <T> void vonArrayNachList (T [] array, List<T> liste) {
2    for (T o : array) {
3      liste.add (o);
4    }
5  }
```

```
1  Cabrio [] cabrioArray = new Cabrio[50];
2  List <Cabrio> cabrioList = new LinkedList<Cabrio> ();
3  vonArrayNachList (cabrioArray, cabrioList);
4      // T ist jetzt vom Typ Cabrio
```

0.8.12.15 Wildcards vs. generische Methoden

- Typparameter bei generischen Methoden gibt Abhängigkeiten zwischen Parametertypen und Rückgabetyp an
- Falls solche Abhängigkeiten nicht existieren, sollten Wildcards verwendet werden

© R.-R. Grigat 2003-2019

Beispiel

```
1 static <T> void vonArrayNachList (T [] array, List<T> liste) {
2   for (T o : array) {
3     liste.add (o);
4   }
5 }
```

- Beide Parameter müssen denselben Typ haben
- Generische Methode benutzen